Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №6
на тему
«ЦИФРОВАЯ ПОДПИСЬ.
ГОСТ Р 34.10-2012»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Электронная цифровая подпись 4
2.2 Параметры и ключи ГОСТ Р 34.10-2012 4
2.3 Формирование подписи 4
2.4 Проверка подписи 5
2.5 Одноразовый множитель *k* 5
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Параметры кривых 6
3.3 Формирование подписи 6
3.4 Проверка подписи 7
3.5 Интерфейс 7
3.6 Тестирование 8
3.7 Замечания по безопасности 9
Заключение 10
Список использованных источников 11
Приложение А (Обязательное) Листинг программного кода 12

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение электронной цифровой подписи и получение практических навыков её формирования и проверки.

В ходе работы требовалось реализовать программное средство формирования и проверки ЭЦП на базе алгоритма ГОСТ Р 34.10-2012:

1 реализовать арифметику эллиптической кривой над простым полем: сложение, удвоение, умножение точки на число и проверку параметров;

2 реализовать формирование ключевой пары, подписи и её проверку по формулам стандарта;

3 воспроизвести тестовый пример из RFC 7091 с точными значениями *r* и *s*;

4 встроить подпись в терминальный интерфейс отдельной вкладкой с режимами Sign и Verify;

5 оформить отчёт и проверить реализацию модульными тестами.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Электронная цифровая подпись

Электронная цифровая подпись – это значение, которое позволяет получателю сообщения убедиться в трёх свойствах [1]:

– подлинности: подпись создал владелец закрытого ключа;

– целостности: сообщение не изменено после подписания;

– неотказуемости: автор не может отказаться от подписи, не раскрыв свой закрытый ключ.

Подписывается не всё сообщение, а его хеш-код. Хеш-функция сжимает сообщение произвольной длины в короткое значение фиксированной длины, поэтому подпись имеет постоянный размер, а проверка не требует повторной обработки всего документа открытым ключом. Стойкость подписи опирается одновременно на стойкость хеш-функции и на сложность задачи дискретного логарифмирования в группе точек эллиптической кривой.

2.2 Параметры и ключи ГОСТ Р 34.10-2012

Схема задаётся эллиптической кривой $y^2 = x^3 + ax + b$ над простым полем $\mathrm{GF}(p)$, базовой точкой G простого порядка q и хеш-функцией ГОСТ Р 34.11-2012 [2].

Закрытый ключ – целое число d, $0 < d < q$. Открытый ключ вычисляется как $Q = dG$. По открытому ключу восстановить d можно только решив задачу дискретного логарифма, которая на кривых достаточно большой длины практически неразрешима.

2.3 Формирование подписи

Для сообщения M вычисляется хеш-код $H = h(M)$, который интерпретируется как целое число α. Затем определяется

$$e = \alpha \bmod q, \quad \text{при } e = 0 \text{ полагается } e = 1. \tag{2.1}$$

Выбирается случайное целое k, $0 < k < q$, вычисляются точка $C = kG$ и числа

$$r = x_C \bmod q, \quad s = (rd + ke) \bmod q. \tag{2.2}$$

Если $r = 0$ или $s = 0$, выбирается новое k и вычисления повторяются. Подпись – пара (r, s); каждое число записывается фиксированным числом байт.

2.4 Проверка подписи

Получатель проверяет условия $0 < r < q$ и $0 < s < q$, вычисляет e по полученному сообщению и находит

$$v = e^{-1} \bmod q, \quad z_1 = sv \bmod q, \quad z_2 = -rv \bmod q. \tag{2.3}$$

Затем строится точка $C = z_1 G + z_2 Q$ и проверяется равенство

$$x_C \bmod q = r. \tag{2.4}$$

Корректность следует из подстановки $Q = dG$:

$$z_1 G + z_2 Q = (s - rd)vG = kevG = kG, \tag{2.5}$$

поскольку $v = e^{-1} \bmod q$. Значит, x_C совпадает с x-координатой точки kG, которая использовалась при подписании.

2.5 Одноразовый множитель k

Значение k должно быть новым для каждой подписи и непредсказуемым. Если две подписи используют одно и то же k, у них совпадает r, и из уравнений

$$s_1 = (rd + ke_1) \bmod q, \quad s_2 = (rd + ke_2) \bmod q \tag{2.6}$$

можно найти $k = (s_1 - s_2)(e_1 - e_2)^{-1} \bmod q$, а затем закрытый ключ $d = (s_1 - ke_1)r^{-1} \bmod q$. Поэтому в реализации k берётся из криптографического генератора случайных чисел; повтор k полностью раскрывает ключ.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Программное средство разделено на четыре уровня:

– *internal/crypto/ec* – арифметика эллиптической кривой: точки, сложение, удвоение, умножение на число, проверка параметров;

– *internal/crypto/gost3410* – формирование ключей, подписи и проверка по формулам ГОСТ Р 34.10-2012;

– *internal/cipher* – интерфейс *SignatureScheme* и адаптер *Gost3410*, который кодирует ключи и подпись в байты фиксированной ширины;

– *internal/ui/signpane* – вкладка *Signature* терминального интерфейса.

Пакет *ec* не содержит ничего, что относилось бы к подписи: им сможет пользоваться и схема Эль-Гамаля на эллиптических кривых. Арифметика реализована в аффинных координатах; бесконечно удалённая точка изображается структурой с нулевыми координатами. Сложение разбирает случаи бесконечно удалённой точки, совпадения точек и взаимно противоположных точек, а умножение использует двоичный метод «удвоение и сложение» [3].

3.2 Параметры кривых

Используются два набора параметров ГОСТ Р 34.10-2012:

– *id-tc26-gost-3410-12-512-paramSetA*: модуль $p = 2^{512} - 569$, 512-битная кривая, порядок q также 512-битный;

– *id-tc26-gost-3410-2012-256-paramSetA*: модуль $p = 2^{256} - 617$, 256-битная кривая с 255-битным простым порядком q и кофактором 4.

В RFC 7836 набор 256-paramSetA приведён в скрученной форме Эдвардса; в работе используется его каноническая короткая форма Вейерштрасса, получаемая бирациональным преобразованием. Оба модуля сравнимы с 3 по модулю 4. Это важно для восстановления y по x извлечением квадратного корня, однако в реализованной схеме такое восстановление не требуется: координаты базовой точки и открытого ключа заданы явно.

Функция *Validate* проверяет, что коэффициенты и координаты лежат в поле, G удовлетворяет уравнению кривой и qG является бесконечно удалённой точкой. Оба набора проходят проверку; тест с изменённым коэффициентом или порядком её не проходит.

3.3 Формирование подписи

Схема формирования подписи показана на рисунке 3.1. Функция *Sign* вычисляет e по хеш-коду, затем в цикле выбирает k и повторяет шаги, пока не получены ненулевые r и s.

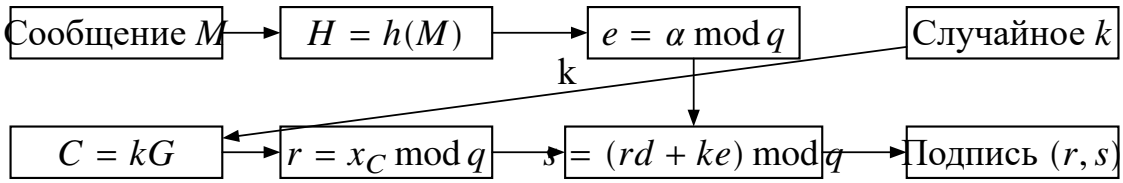


Рисунок 3.1 – Схема формирования подписи ГОСТ Р 34.10-2012

Если $r = 0$ или $s = 0$, выбирается новое k; вероятность такого события пренебрежимо мала, но цикл обеспечивает соответствие стандарту. Подпись кодируется как конкатенация r и s, по 32 байта для 256-битной кривой и по 64 байта для 512-битной.

3.4 Проверка подписи

Проверка показана на рисунке 3.2. Сначала отвергаются r и s вне диапазона $1..q - 1$; это штатный отрицательный результат, а не ошибка программы. Затем вычисляются v, z_1, z_2, точка C и сравнивается $x_C \bmod q$ с r.

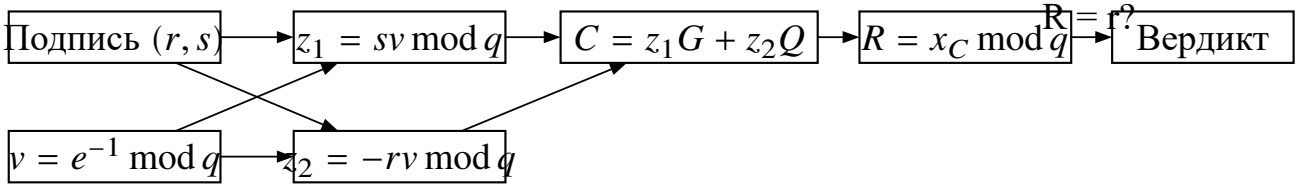


Рисунок 3.2 – Схема проверки подписи ГОСТ Р 34.10-2012

Вердикт *ПОДПИСЬ ВЕРНА* выводится при совпадении R и r; иначе выводится *ПОДПИСЬ НЕВЕРНА* с указанием $R \neq r$. Проверка не изменяет поле подписи, поэтому одну и ту же подпись можно проверить для разных сообщений и ключей.

3.5 Интерфейс

Вкладка *Signature* построена по образцу асимметричной панели и состоит из трёх полос. Верхняя полоса – кольцо ключей: выбор кривой (256 или 512 бит), открытый ключ, закрытый ключ (скрыт, *Ctrl+R* показывает его, *Ctrl+K* очищает) и кнопка *GENERATE KEYPAIR* (*Ctrl+G*).

Средняя полоса содержит сообщение слева и подпись справа. В режиме Sign подпись является выходом, в режиме Verify – редактируемым входом; под полем подписи выводится строка вердикта. Источник данных переключается между ручным вводом и файлом. Кнопка *RUN* формирует или проверяет подпись, *SAVE TO FILE* сохраняет её в файл.

Клавиша *Ctrl+D* оставляет подпись в поле и переключает панель в режим проверки, а *Alt+D* очищает сообщение. Пустой закрытый ключ в режиме Sign,

пустой открытый ключ или неразбираемая подпись в режиме Verify дают строку ошибки, а не панику.

3.6 Тестирование

Корректность проверена командой `go test ./...`. Все тесты проходят.

Главный тест *TestRFC7091Example* воспроизводит пример из раздела 7 RFC 7091. В RFC приведены параметры кривой, закрытый ключ d, значение e, множитель k и ожидаемые r и s (таблица 3.1). Сообщение, по которому получено e, в RFC не указано, поэтому тест подаёт готовое e через подменную хеш-функцию и фиксирует k; формулы при этом выполняются без упрощений.

Таблица 3.1 – Тестовый пример RFC 7091

Величина	Значение
p	8000000000000000000000000000000000000000000000000000000000000431
q	8000000000000000000000000000000150FE8A1892976154C59CFC193ACCF5B3
d	7A929ADE789BB9BE10ED359DD39A72C11B60961F49397EEE1D19CE9891EC3B28
k	77105C9B20BCD3122823C8CF6FCC7B956DE33814E95B7FE64FED924594DCEAB3
e	2DFBC1B372D89A1188C09C52E0EEC61FCE52032AB1022E8E67ECE6672B043EE5
r	41AA28D2F1AB148280CD9ED56FEDA41974053554A42767B83AD043FD39DC0493
s	1456C64BA4642A1653C235A98A60249BCD6D3F746B631DF928014F6C5BF9C40

Полученные r и s совпадают с ожидаемыми, а точка $Q = dG$ – с координатами открытого ключа из RFC. Дополнительно проверены:

– *TestSignVerifyRoundTrip* – цикл «подпись–проверка» на обеих кривых для пустого, текстового и двоичного сообщения;

– *TestTamperedInputsRejected* – изменённое сообщение, повреждённое s и чужой открытый ключ отвергаются;

– *TestOutOfRangeSignatureRejected* – значения r или s, равные нулю, q и $q + 1$, отвергаются;

– *TestDeterministicNonce* – при одном и том же k подпись повторяется, что подтверждает формулу s;

– *TestParamSetsValidate* и *TestTamperedParametersRejected* – проверка параметров обеих кривых и отказ при повреждённом коэффициенте или порядке.

3.7 Замечания по безопасности

– Множитель k берётся из *crypto/rand* и не повторяется; ошибка чтения генератора возвращается вызывающему коду.

– Проверяется диапазон r и s, открытый ключ должен лежать на кривой и иметь порядок q; бесконечно удалённая точка отвергается.

– Длина ключей и подписи фиксирована: 32 или 64 байта на число. Неверная длина – ошибка разбора, а не попытка угадать параметры.

– Реализация учебная: *math/big* не даёт постоянного времени вычислений, поэтому она не защищена от атак по времени и побочным каналам. Для промышленного применения нужна константная арифметика и проверенная библиотека.

– Подпись хранится как простое $r \parallel s$ без ASN.1-кодирования; формат обмена с другими системами должен быть согласован отдельно.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована электронная цифровая подпись ГОСТ Р 34.10-2012. Арифметика эллиптической кривой вынесена в отдельный пакет, который не зависит от схемы подписи; поверх него построены формирование ключей, подписи и проверка по формулам стандарта. Подпись встроена в терминальный интерфейс отдельной вкладкой с режимами Sign и Verify, выбором 256- и 512-битной кривой, ручным вводом или чтением файла и наглядной строкой вердикта.

Корректность подтверждена тестовым примером RFC 7091 с точным совпадением Q, r и s, а также тестами на цикл «подпись–проверка», повреждённые данные, чужие ключи и граничные значения. Показано, что повторное использование одноразового множителя k раскрывает закрытый ключ, поэтому генерация k доверена криптографическому источнику случайности.

Ограничением учебной реализации является отсутствие константного времени вычислений; для реальных систем требуется проверенная криптографическая библиотека.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Dolmatov, V. GOST R 34.10-2012: Digital Signature Algorithm / V. Dolmatov, A. Degtyarev. – 2013.

[2] Smyshlyaev, S. Cryptographic Algorithms for the GOST R 34.10-2012 and GOST R 34.11-2012 Standards / S. Smyshlyaev [и др.]. – 2016.

[3] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

[4] Rabin, M.O. Digitalized Signatures and Public-Key Functions as Intractable as Factorization / M.O. Rabin. – 1979.

[5] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

[6] Dolmatov, V. GOST R 34.11-2012: Hash Function / V. Dolmatov, A. Degtyarev. – 2013.

[7] Technology. Secure Hash Standard (SHS) / National Institute of Standards and Technology. – 2015.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Арифметика эллиптической кривой

```
// Пакет ec реализует арифметику точек эллиптической кривой над простым
// полем в короткой форме Вейерштрасса
//
//  y^2 = x^3 + a*x + b (mod p).
//
// Точки хранятся в аффинных координатах. Бесконечно удалённая точка
// изображается значением Point{} — обе координаты nil. Пакет не содержит
// ничего, что относилось бы к конкретной схеме подписи: им пользуются и
// ГОСТ Р 34.10-2012, и схема Эль-Гамаля на эллиптических кривых.
package ec

import (
  "errors"
  "fmt"
  "math/big"
)

var (
  // ErrInvalidParams — набор параметров кривой не прошёл проверку.
  ErrInvalidParams = errors.New("ec: invalid curve parameters")
  // ErrPointNotOnCurve — точка не лежит на кривой.
  ErrPointNotOnCurve = errors.New("ec: point is not on the curve")
)

// Point — точка кривой в аффинных координатах. Если X и Y равны nil,
// точка бесконечно удалённая.
type Point struct {
  X, Y *big.Int
}

// IsInfinity сообщает, является ли точка бесконечно удалённой.
func (p Point) IsInfinity() bool {
  return p.X == nil && p.Y == nil
}

// Equal сравнивает две точки.
func (p Point) Equal(q Point) bool {
  if p.IsInfinity() || q.IsInfinity() {
    return p.IsInfinity() && q.IsInfinity()
  }
  return p.X.Cmp(q.X) == 0 && p.Y.Cmp(q.Y) == 0
}

// Curve — параметры эллиптической кривой и порядок N базовой точки.
//
//  GX, GY — координаты базовой точки G;
//  P      — модуль поля (простое число);
//  A, B   — коэффициенты уравнения кривой;
//  N      — порядок точки G (простое число).
type Curve struct {
  Name string
  P    *big.Int
  A    *big.Int
  B    *big.Int
  N    *big.Int
  GX   *big.Int
```

```
  GY   *big.Int
}

// Base возвращает базовую точку кривой.
func (c *Curve) Base() Point {
  return Point{X: c.GX, Y: c.GY}
}

// Add складывает точки P и Q по правилу секущей:
//
//  lambda = (y2 - y1) / (x2 - x1) (mod p),
//  x3 = lambda^2 - x1 - x2,
//  y3 = lambda*(x1 - x3) - y1.
//
// Особые случаи: бесконечно удалённая точка — нейтральный элемент,
// а сумма точки и ей противоположной — бесконечно удалённая точка.
func (c *Curve) Add(p, q Point) Point {
  if p.IsInfinity() {
    return q
  }
  if q.IsInfinity() {
    return p
  }

  if p.X.Cmp(q.X) == 0 {
    // При одинаковых x точки либо совпадают (удвоение), либо
    // являются взаимно противоположными.
    if p.Y.Cmp(q.Y) != 0 {
      return Point{}
    }
    return c.Double(p)
  }

  // lambda = (y2 - y1) / (x2 - x1) mod p.
  num := new(big.Int).Sub(q.Y, p.Y)
  den := new(big.Int).Sub(q.X, p.X)
  den.ModInverse(den, c.P)
  if den == nil {
    // Обратного элемента нет только при составном p; такая кривая
    // отвергается в Validate.
    return Point{}
  }
  lambda := new(big.Int).Mul(num, den)
  lambda.Mod(lambda, c.P)

  x3 := new(big.Int).Mul(lambda, lambda)
  x3.Sub(x3, p.X)
  x3.Sub(x3, q.X)
  x3.Mod(x3, c.P)

  y3 := new(big.Int).Sub(p.X, x3)
  y3.Mul(lambda, y3)
  y3.Sub(y3, p.Y)
  y3.Mod(y3, c.P)

  return Point{X: x3, Y: y3}
}

// Double удваивает точку по правилу касательной:
//
//  lambda = (3*x^2 + a) / (2*y) (mod p),
//  x3 = lambda^2 - 2*x,
//  y3 = lambda*(x - x3) - y.
```

```
//
// Точка с y = 0 имеет порядок 2, поэтому её удвоение — бесконечно
// удалённая точка.
func (c *Curve) Double(p Point) Point {
  if p.IsInfinity() || p.Y.Sign() == 0 {
    return Point{}
  }

  // lambda = (3*x^2 + a) / (2*y) mod p.
  num := new(big.Int).Mul(p.X, p.X)
  num.Mul(num, big.NewInt(3))
  num.Add(num, c.A)
  num.Mod(num, c.P)

  den := new(big.Int).Lsh(p.Y, 1)
  den.ModInverse(den, c.P)
  if den == nil {
    return Point{}
  }
  lambda := new(big.Int).Mul(num, den)
  lambda.Mod(lambda, c.P)

  x3 := new(big.Int).Mul(lambda, lambda)
  x3.Sub(x3, p.X)
  x3.Sub(x3, p.X)
  x3.Mod(x3, c.P)

  y3 := new(big.Int).Sub(p.X, x3)
  y3.Mul(lambda, y3)
  y3.Sub(y3, p.Y)
  y3.Mod(y3, c.P)

  return Point{X: x3, Y: y3}
}

// Neg возвращает точку, противоположную данной: (x, -y mod p).
func (c *Curve) Neg(p Point) Point {
  if p.IsInfinity() {
    return Point{}
  }
  y := new(big.Int).Neg(p.Y)
  y.Mod(y, c.P)
  return Point{X: new(big.Int).Set(p.X), Y: y}
}

// ScalarMult вычисляет k*P методом «удвоение и сложение»: двоичное
// представление k просматривается от младшего бита к старшему, текущая
// точка при каждом шаге удваивается, а при единичном бите добавляется к
// результату. Для отрицательного k вычисляется |k|*(-P).
func (c *Curve) ScalarMult(k *big.Int, p Point) Point {
  if k == nil || k.Sign() == 0 || p.IsInfinity() {
    return Point{}
  }

  scalar := k
  if scalar.Sign() < 0 {
    scalar = new(big.Int).Neg(scalar)
    p = c.Neg(p)
  }

  result := Point{}
  addend := p
  for i := 0; i < scalar.BitLen(); i++ {
```

```
    if scalar.Bit(i) == 1 {
      result = c.Add(result, addend)
    }
    addend = c.Double(addend)
  }
  return result
}

// IsOnCurve проверяет уравнение y^2 = x^3 + a*x + b (mod p) и то, что
// координаты приведены по модулю p. Бесконечно удалённая точка считается
// лежащей на кривой.
func (c *Curve) IsOnCurve(p Point) bool {
  if p.IsInfinity() {
    return true
  }
  if p.X == nil || p.Y == nil {
    return false
  }
  if p.X.Sign() < 0 || p.X.Cmp(c.P) >= 0 || p.Y.Sign() < 0 || p.Y.Cmp(c.P) >=
0 {
    return false
  }

  left := new(big.Int).Mul(p.Y, p.Y)
  left.Mod(left, c.P)

  right := new(big.Int).Mul(p.X, p.X)
  right.Mul(right, p.X)
  right.Add(right, new(big.Int).Mul(c.A, p.X))
  right.Add(right, c.B)
  right.Mod(right, c.P)

  return left.Cmp(right) == 0
}

// Validate проверяет, что параметры кривой корректны: коэффициенты и
// координаты базовой точки лежат в поле, G удовлетворяет уравнению
// кривой и порядок N действительно аннулирует G (N*G — бесконечно
// удалённая точка).
func (c *Curve) Validate() error {
  if c == nil {
    return fmt.Errorf("%w: curve is nil", ErrInvalidParams)
  }
  if c.P == nil || c.A == nil || c.B == nil || c.N == nil || c.GX == nil ||
c.GY == nil {
    return fmt.Errorf("%w: missing parameters", ErrInvalidParams)
  }
  if c.P.Sign() <= 0 || c.P.Bit(0) == 0 {
       return fmt.Errorf("%w: modulus must be an odd positive number",
ErrInvalidParams)
  }
  if c.A.Sign() < 0 || c.A.Cmp(c.P) >= 0 {
    return fmt.Errorf("%w: coefficient a out of range", ErrInvalidParams)
  }
  if c.B.Sign() < 0 || c.B.Cmp(c.P) >= 0 {
    return fmt.Errorf("%w: coefficient b out of range", ErrInvalidParams)
  }
  if c.N.Cmp(big.NewInt(1)) <= 0 {
    return fmt.Errorf("%w: order must be greater than one", ErrInvalidParams)
  }
  if !c.IsOnCurve(c.Base()) {
    return fmt.Errorf("%w: base point", ErrPointNotOnCurve)
  }
```

```
  if !c.ScalarMult(c.N, c.Base()).IsInfinity() {
    return fmt.Errorf("%w: N*G is not the point at infinity", ErrInvalidParams)
  }
  return nil
}

// clone возвращает независимую копию параметров, чтобы вызывающий код не
// мог изменить общие big.Int.
func (c *Curve) clone() *Curve {
  return &Curve{
    Name: c.Name,
    P:    new(big.Int).Set(c.P),
    A:    new(big.Int).Set(c.A),
    B:    new(big.Int).Set(c.B),
    N:    new(big.Int).Set(c.N),
    GX:   new(big.Int).Set(c.GX),
    GY:   new(big.Int).Set(c.GY),
  }
}
```

Листинг А.2 – Параметры кривых ГОСТ Р 34.10-2012

```
package ec

import "math/big"

// Параметры эллиптических кривых ГОСТ Р 34.10-2012. Значения записаны в
// шестнадцатеричном виде, старшим байтом вперёд, как в RFC 7836.

// params256 — id-tc26-gost-3410-2012-256-paramSetA
// (OID 1.2.643.7.1.2.1.1.1). В RFC 7836, прил. А.2 эта кривая задана в
// скрученной форме Эдвардса; приведённые ниже p, a, b, n и G — её
// каноническая короткая форма Вейерштрасса, полученная бирациональным
// преобразованием. Порядок n — простое число, кофактор равен 4.
var params256 = &Curve{
  Name: "ГОСТ 34.10-2012 · 256",
                                                                    P:
hexInt("FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFD97"),
                                                                    A:
hexInt("C2173F1513981673AF4892C23035A27CE25E2013BF95AA33B22C656F277E7335"),
                                                                    B:
hexInt("295F9BAE7428ED9CCC20E7C359A9D41A22FCCD9108E17BF7BA9337A6F8AE9513"),
                                                                    N:
hexInt("400000000000000000000000000000FD8CDDFC87B6635C115AF556C360C67"),
                                                                   GX:
hexInt("91E38443A5E82C0D880923425712B2BB658B9196932E02C78B2582FE742DAA28"),
                                                                   GY:
hexInt("32879423AB1A0375895786C4BB46E9565FDE0B5344766740AF268ADB32322E5C"),
}

// params512 — id-tc26-gost-3410-12-512-paramSetA
// (OID 1.2.643.7.1.2.1.2.1), RFC 7836, прил. А.1.
var params512 = &Curve{
  Name: "ГОСТ 34.10-2012 · 512",

P:   hexInt("FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
+ "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFDC7"),

A:   hexInt("FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
+ "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFDC4"),

B:   hexInt("E8C2505DEDFC86DDC1BD0B2B6667F1DA34B82574761CB0E879BD081CFD0B6265"
+ "EE3CB090F30D27614CB4574010DA90DD862EF9D4EBEE4761503190785A71C760"),
```

```
N:   hexInt("FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
+ "27E69532F48D89116FF22B8D4E0560609B4B38ABFAD2B85DCACDB1411F10B275"),
  GX:   big.NewInt(3),
                                                                    GY:
hexInt("7503CFE87A836AE3A61B8816E25450E6CE5E1C93ACF1ABC1778064FDCBEFA921"    +
"DF1626BE4FD036E93D75E6A50E3A41E98028FE5FC235F5B889A589CB5215F2A4"),
}

// Params256 возвращает параметры 256-битной кривой.
func Params256() *Curve {
  return params256.clone()
}

// Params512 возвращает параметры 512-битной кривой.
func Params512() *Curve {
  return params512.clone()
}

// hexInt разбирает шестнадцатеричную запись числа.
func hexInt(s string) *big.Int {
  n, ok := new(big.Int).SetString(s, 16)
  if !ok {
    panic("ec: invalid hexadecimal constant " + s)
  }
  return n
}
```

Листинг А.3 – Формирование и проверка подписи

```
// Пакет gost3410 реализует формирование и проверку электронной цифровой
// подписи по ГОСТ Р 34.10-2012 (RFC 7091).
//
// Обозначения стандарта: q — порядок базовой точки G, d — закрытый ключ,
// Q = d*G — открытый ключ, e — свёртка хеш-кода сообщения по модулю q,
// k — одноразовый случайный множитель, (r, s) — подпись.
package gost3410

import (
  "crypto/rand"
  "errors"
  "fmt"
  "hash"
  "io"
  "math/big"

  "go-cipher/internal/crypto/ec"
)

var (
  // ErrPrivateKey — закрытый ключ не лежит в диапазоне 1..q-1.
  ErrPrivateKey = errors.New("gost3410: private key out of range")
  // ErrPublicKey — открытый ключ не является точкой кривой порядка q.
  ErrPublicKey = errors.New("gost3410: invalid public key")
  // ErrNonce — не удалось получить случайное k.
  ErrNonce = errors.New("gost3410: cannot generate nonce")
  // ErrHash — не задана хеш-функция.
  ErrHash = errors.New("gost3410: hash function is not set")
)

var (
  one = big.NewInt(1)
)
```

```
// GenerateKey формирует ключевую пару: случайный закрытый ключ
// d из диапазона 1..q-1 и открытый ключ Q = d*G.
func GenerateKey(c *ec.Curve) (*big.Int, ec.Point, error) {
  if err := c.Validate(); err != nil {
    return nil, ec.Point{}, err
  }

  // d = random(1..q-1).
  limit := new(big.Int).Sub(c.N, one)
  d, err := rand.Int(rand.Reader, limit)
  if err != nil {
   return nil, ec.Point{}, fmt.Errorf("gost3410: generate private key: %w", err)
  }
  d.Add(d, one)

  return d, c.ScalarMult(d, c.Base()), nil
}

// Sign вычисляет подпись (r, s) сообщения msg.
//
// Шаги алгоритма I стандарта:
//  1. H = h(msg), e = int(H) mod q (при e = 0 берётся e = 1);
//  2. выбирается случайное k из 1..q-1;
//  3. C = k*G, r = x_C mod q; при r = 0 выбирается новое k;
//  4. s = (r*d + k*e) mod q; при s = 0 выбирается новое k.
func Sign(c *ec.Curve, hashAlg func() hash.Hash, priv *big.Int, msg []byte, rnd
io.Reader) (*big.Int, *big.Int, error) {
  if c == nil || c.N == nil || c.N.Cmp(one) <= 0 {
    return nil, nil, ec.ErrInvalidParams
  }
  if priv == nil || priv.Sign() <= 0 || priv.Cmp(c.N) >= 0 {
    return nil, nil, ErrPrivateKey
  }

  e, err := hashToE(c, hashAlg, msg)
  if err != nil {
    return nil, nil, err
  }

  for {
    k, err := randomK(c.N, rnd)
    if err != nil {
      return nil, nil, err
    }

    r, s, ok := signWithNonce(c, priv, e, k)
    if ok {
      return r, s, nil
    }
  }
}

// Verify проверяет подпись (r, s) сообщения msg открытым ключом pub.
//
// Шаги алгоритма II стандарта:
//  1. проверяется 0 < r < q и 0 < s < q;
//  2. e вычисляется так же, как при подписи, v = e^{-1} mod q;
//  3. z1 = s*v mod q, z2 = -r*v mod q;
//  4. C = z1*G + z2*Q; подпись верна, если x_C mod q = r.
//
// Для корректной подписи возвращается true. Математически неверная
// подпись, в том числе с r или s вне диапазона, даёт false без ошибки:
```

```
// это штатный результат проверки, а не сбой.
func Verify(c *ec.Curve, hashAlg func() hash.Hash, pub ec.Point, msg []byte, r,
s *big.Int) (bool, error) {
  if c == nil || c.N == nil || c.N.Cmp(one) <= 0 {
    return false, ec.ErrInvalidParams
  }
      if   pub.IsInfinity()   ||   !c.IsOnCurve(pub)   ||   !c.ScalarMult(c.N,
pub).IsInfinity() {
    return false, ErrPublicKey
  }
  if r == nil || s == nil || r.Sign() <= 0 || s.Sign() <= 0 || r.Cmp(c.N) >= 0
|| s.Cmp(c.N) >= 0 {
    return false, nil
  }

  e, err := hashToE(c, hashAlg, msg)
  if err != nil {
    return false, err
  }

  // v = e^{-1} mod q.
  v := new(big.Int).ModInverse(e, c.N)
  if v == nil {
    return false, fmt.Errorf("gost3410: e has no inverse modulo q")
  }

  // z1 = s*v mod q, z2 = -r*v mod q.
  z1 := new(big.Int).Mul(s, v)
  z1.Mod(z1, c.N)

  z2 := new(big.Int).Mul(r, v)
  z2.Neg(z2)
  z2.Mod(z2, c.N)

  // C = z1*G + z2*Q.
  cPoint := c.Add(c.ScalarMult(z1, c.Base()), c.ScalarMult(z2, pub))
  if cPoint.IsInfinity() {
    return false, nil
  }

  // R = x_C mod q.
  bigR := new(big.Int).Mod(cPoint.X, c.N)
  return bigR.Cmp(r) == 0, nil
}

// signWithNonce выполняет шаги 3-4 алгоритма I для уже выбранных e и k.
// ok = false означает, что получено r = 0 или s = 0 и нужно новое k.
func signWithNonce(c *ec.Curve, priv, e, k *big.Int) (r, s *big.Int, ok bool) {
  cPoint := c.ScalarMult(k, c.Base())
  if cPoint.IsInfinity() {
    return nil, nil, false
  }

  r = new(big.Int).Mod(cPoint.X, c.N)
  if r.Sign() == 0 {
    return nil, nil, false
  }

  // s = (r*d + k*e) mod q.
  s = new(big.Int).Mul(r, priv)
  s.Add(s, new(big.Int).Mul(k, e))
  s.Mod(s, c.N)
  if s.Sign() == 0 {
```

```
    return nil, nil, false
  }

  return r, s, true
}

// hashToE вычисляет e = int(h(msg)) mod q, заменяя нулевой результат на 1.
func hashToE(c *ec.Curve, hashAlg func() hash.Hash, msg []byte) (*big.Int,
error) {
  if hashAlg == nil {
    return nil, ErrHash
  }

  h := hashAlg()
  h.Write(msg)
  e := new(big.Int).SetBytes(h.Sum(nil))
  e.Mod(e, c.N)
  if e.Sign() == 0 {
    e.SetInt64(1)
  }
  return e, nil
}

// randomK выбирает k из диапазона 1..q-1, читая байты из rnd.
func randomK(q *big.Int, rnd io.Reader) (*big.Int, error) {
  for {
    k, err := rand.Int(rnd, q)
    if err != nil {
      return nil, fmt.Errorf("%w: %v", ErrNonce, err)
    }
    if k.Sign() != 0 {
      return k, nil
    }
  }
}
```